

PLACEMENT PREPARATION TRACKER

A Full-Stack CRUD-Based Web Application

Project Documentation

Developed by: Mahasakthi Manikandan

Programme: B.Tech Information Technology

Project Type: Academic Full-Stack Web Application

Technologies: React • Spring Boot • MySQL • REST API • Git • GitHub

TABLE OF CONTENTS

1. Project Overview
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Technology Stack
6. System Architecture
7. Project Modules
8. Database Design and ER Diagram
9. CRUD Implementation
10. REST API Documentation
11. Frontend Implementation
12. Backend Implementation
13. Validation and Exception Handling
14. Testing and Results
15. Installation and Execution
16. Project Structure
17. Challenges and Solutions
18. Git and GitHub
19. Future Enhancements
20. Results and Benefits
21. Conclusion

1. PROJECT OVERVIEW

Placement Preparation Tracker is a full-stack web application developed to help students manage and track their placement preparation activities in one centralized platform.

The application allows students to maintain company information, job applications, preparation topics, and interview records. It also provides a dashboard that displays important placement statistics and overall preparation progress.

The frontend of the application is developed using React. The backend is developed using Java Spring Boot and provides REST APIs. MySQL is used as the database for permanent data storage.

The application implements Create, Read, Update, and Delete operations for the major modules and includes validation, exception handling, search, filtering, testing, and database integration.

2. PROBLEM STATEMENT

Students preparing for placements often maintain company details, job applications, preparation notes, and interview information separately.

Managing these details manually can make it difficult to:

- Track applied companies.
- Monitor application status.
- Maintain preparation progress.
- Record interview information.
- Search previous records.
- Review overall placement progress.

The Placement Preparation Tracker solves this problem by providing a centralized web application where all placement-related information can be managed in one place.

3. OBJECTIVES

The main objectives of the project are:

1. To develop a centralized placement preparation tracking system.
2. To maintain company information.
3. To manage job application details.
4. To track preparation topics and progress.
5. To maintain interview records and feedback.
6. To implement complete CRUD functionality.
7. To integrate a React frontend with a Spring Boot backend.
8. To store application data in a MySQL database.
9. To implement input validation and exception handling.
10. To test the REST APIs using Postman.
11. To use Git and GitHub for version control and project submission.

The SOP defines the objective of the CRUD application as implementing a functional frontend, backend, REST API, database, validation, testing, and documentation.

4. SCOPE OF THE PROJECT

The project is designed for students who want to organize their placement preparation.

The current application provides four major modules:

- Company Management
- Application Management
- Preparation Management

- Interview Management

The system supports creating, viewing, editing, deleting, searching, and filtering relevant records.

The dashboard provides a quick overview of the student's placement preparation status.

The project can later be extended with authentication, reminders, analytics, resume management, and other placement-related features.

5. TECHNOLOGY STACK

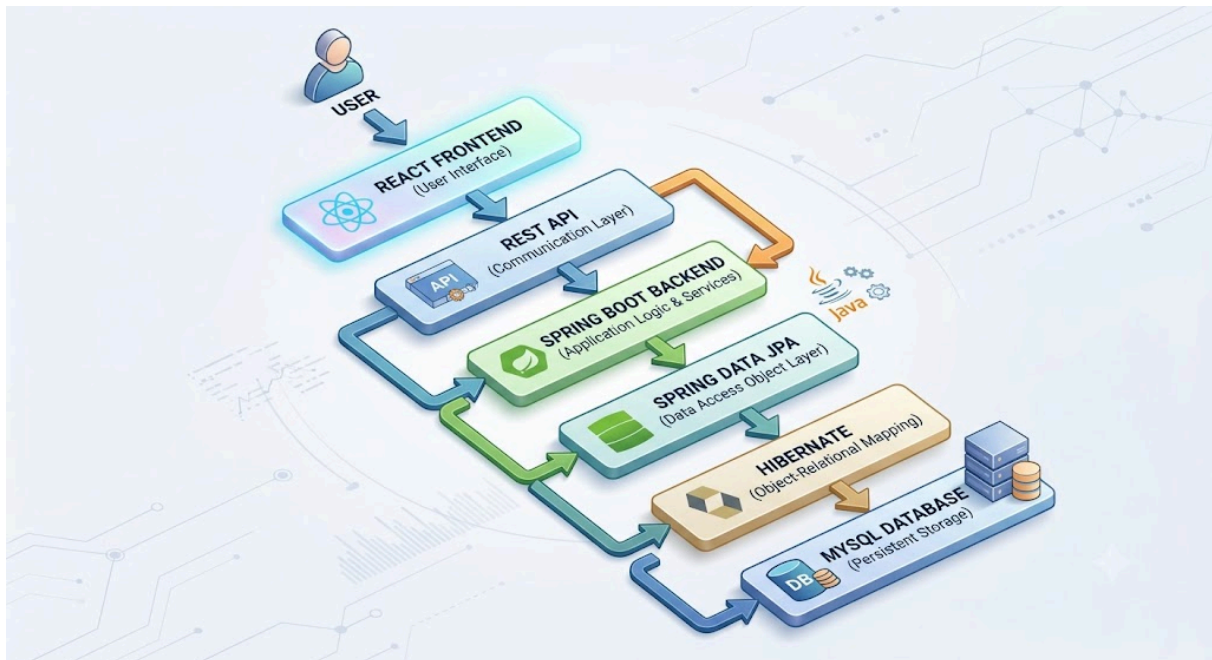
Layer	Technology	Purpose
Frontend	React	User interface
Frontend Tools	HTML, CSS, JavaScript, Vite	UI development and interaction
Backend	Java, Spring Boot	REST API and server-side logic
ORM	Spring Data JPA, Hibernate	Database communication
Validation	Jakarta Validation	Input validation
Database	MySQL	Permanent data storage
API Testing	Postman	REST API testing
Database Tool	MySQL Workbench	Database verification
Version Control	Git	Source-code management
Repository	GitHub	Remote source-code hosting
IDE	Visual Studio Code	Development

The selected technologies fit the SOP's recommended stack, which allows React for the frontend, Spring Boot as a backend alternative, MySQL as a database, Postman for API testing, and Git/GitHub for version control.

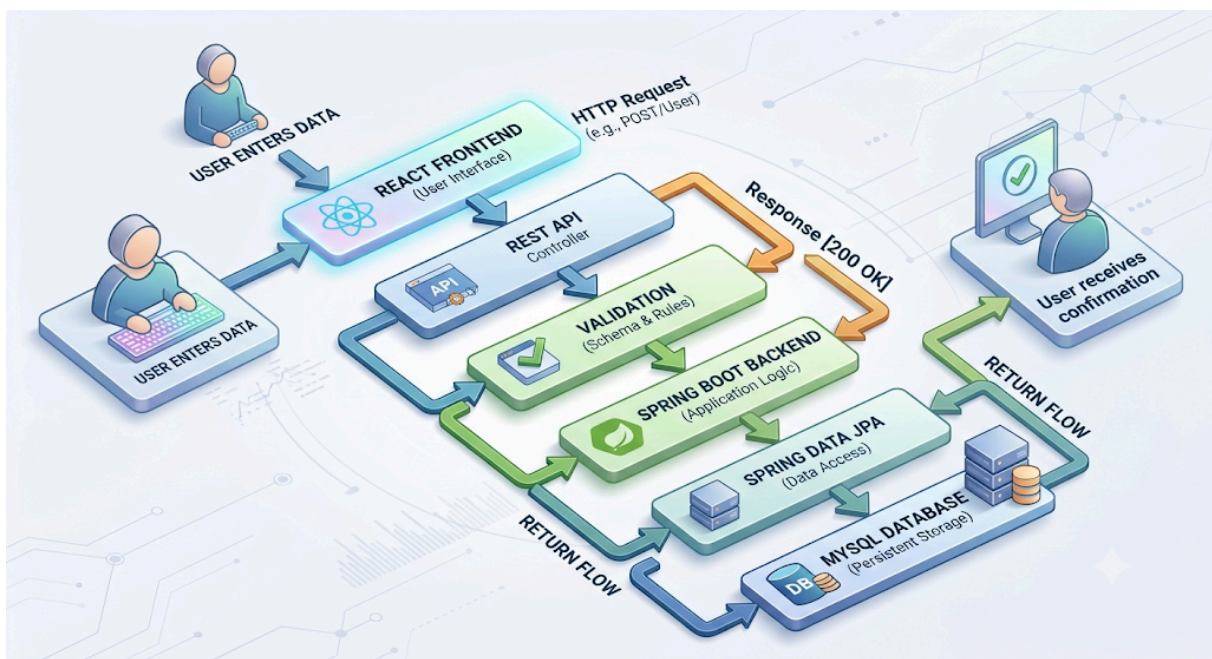
6. SYSTEM ARCHITECTURE

6.1 Architecture

The application follows a client-server architecture.



6.2 Data Flow



The architecture follows the SOP's suggested flow from frontend through REST API and Spring Boot to ORM/JPA and the database.

7. PROJECT MODULES

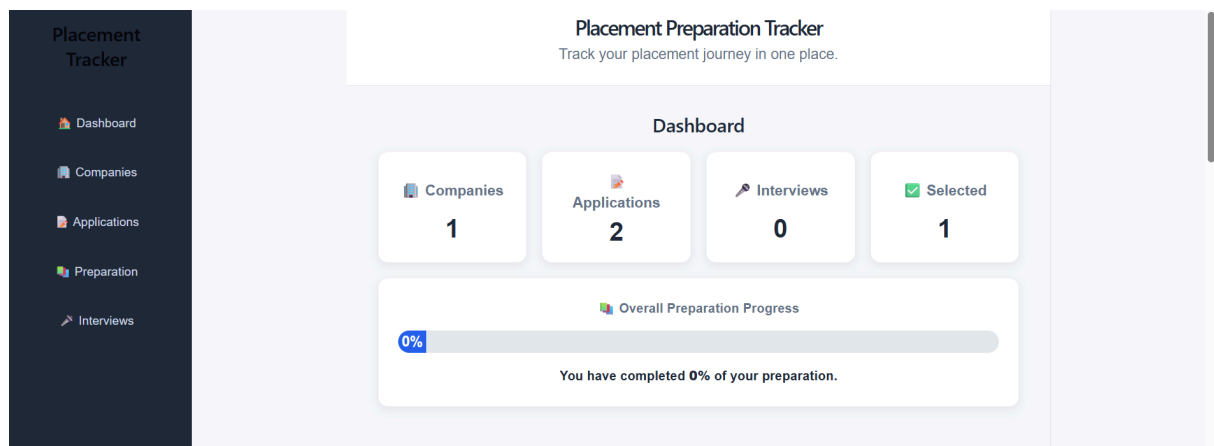
7.1 Dashboard

The Dashboard provides a summary of placement preparation.

It displays:

- Total Companies
- Total Applications
- Total Interviews
- Selected Applications
- Overall Preparation Progress

The dashboard helps the student understand the current placement preparation status quickly.



7.2 Company Management

The Company Management module allows users to maintain company information.

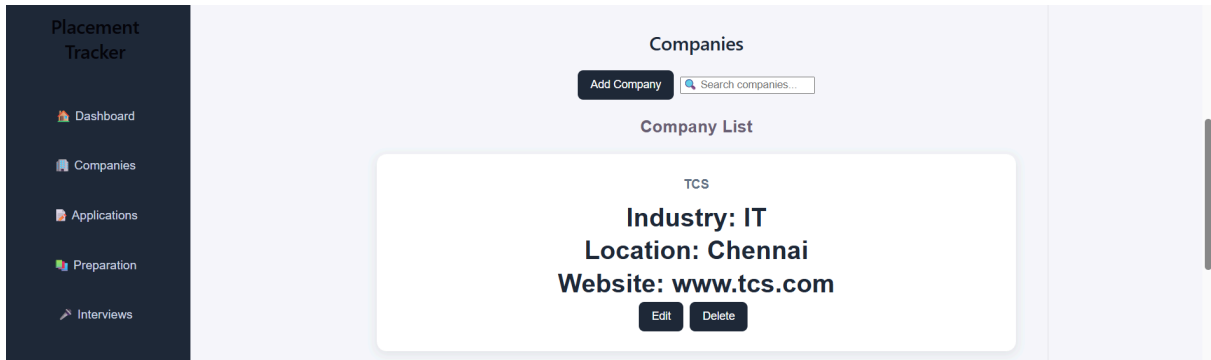
Fields

- Company Name
- Industry
- Location
- Website

Operations

- Add Company
- View Companies
- Edit Company

- Delete Company
- Search Company



7.3 Application Management

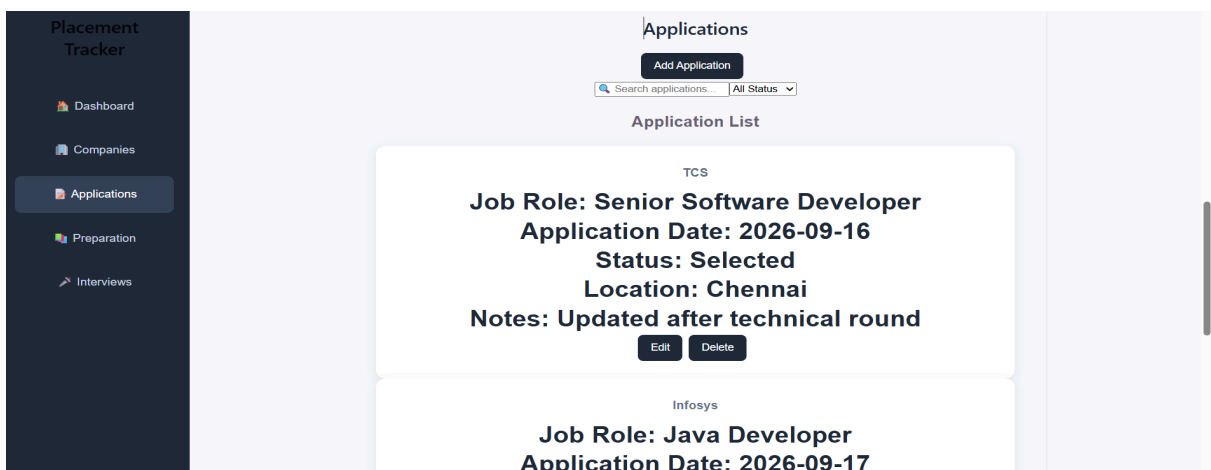
The Application Management module stores job application information.

Fields

- Company
- Job Role
- Application Date
- Status
- Location
- Notes

Operations

- Add Application
- View Applications
- Edit Application
- Delete Application
- Search Applications
- Filter Applications by Status



7.4 Preparation Management

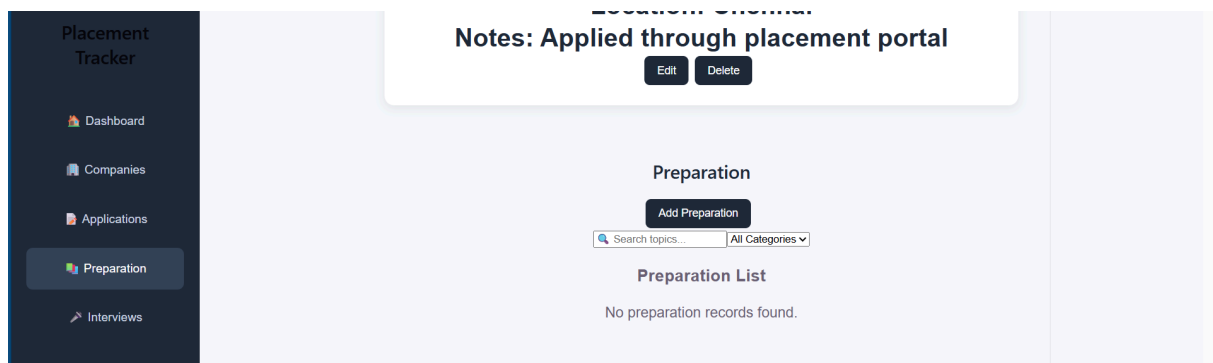
The Preparation module helps students track technical and other preparation topics.

Fields

- Topic
- Category
- Progress
- Status
- Notes

Operations

- Add Preparation
- View Preparation
- Edit Preparation
- Delete Preparation
- Search Topics
- Filter by Category
- Track Progress



7.5 Interview Management

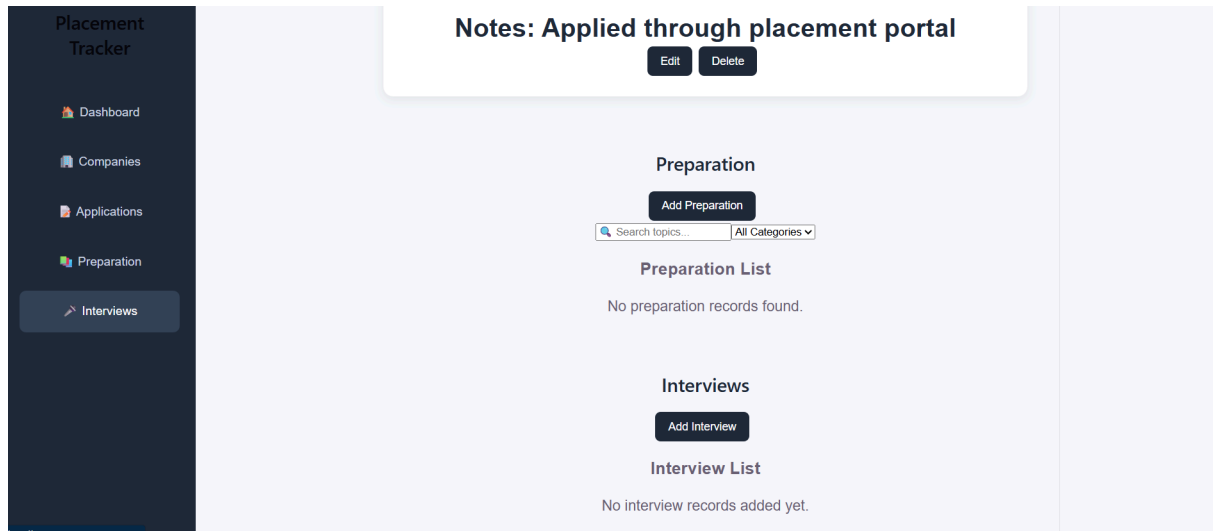
The Interview Management module stores interview information.

Fields

- Company
- Round Name
- Interview Date
- Result
- Feedback

Operations

- Add Interview
- View Interviews
- Edit Interview
- Delete Interview



8. DATABASE DESIGN AND ER DIAGRAM

8.1 Database

The project uses MySQL.

Database Name

placement_tracker

The database contains the following major tables:

company
application
preparation
interview

8.2 Company Table

Field	Description
id	Primary Key
company_name	Company name
industry	Industry
location	Company location
website	Company website

8.3 Application Table

Field	Description
id	Primary Key
company	Company name
job_role	Applied job role
application_date	Date of application
status	Application status
location	Job location
notes	Additional notes

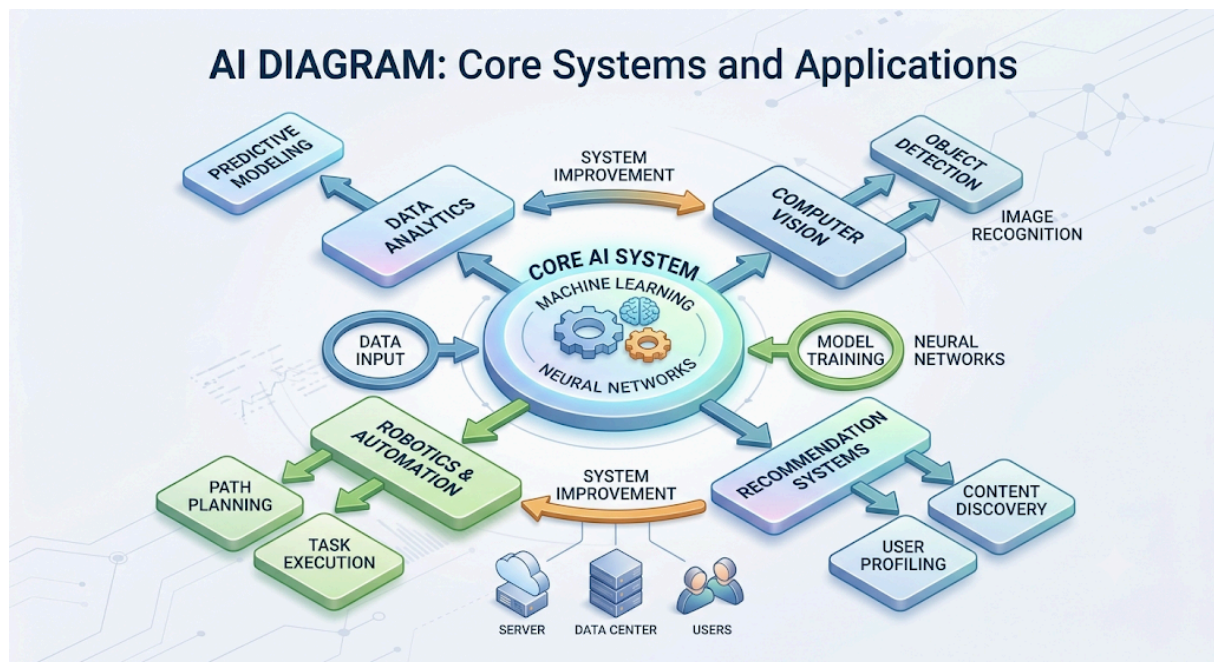
8.4 Preparation Table

Field	Description
id	Primary Key
topic	Preparation topic
category	Topic category
progress	Completion percentage
status	Preparation status
notes	Additional notes

8.5 Interview Table

Field	Description
id	Primary Key
company	Company name
round_name	Interview round
interview_date	Interview date
result	Interview result
feedback	Interview feedback

8.6 ER Diagram



9. CRUD IMPLEMENTATION

CRUD stands for:

- **Create**
- **Read**
- **Update**
- **Delete**

The application implements CRUD functionality for all four major modules.

9.1 Company CRUD

Create: Add a new company.

Read: Display company records.

Update: Modify company details.

Delete: Remove a company record.

9.2 Application CRUD

Create: Add a job application.

Read: Display applications.

Update: Modify application information.

Delete: Remove an application.

9.3 Preparation CRUD

Create: Add a preparation topic.

Read: Display preparation topics.

Update: Modify topic details and progress.

Delete: Remove a preparation record.

9.4 Interview CRUD

Create: Add an interview.

Read: Display interview records.

Update: Modify interview information.

Delete: Remove an interview.

The SOP requires all four CRUD operations to be implemented and demonstrated correctly.

10. REST API DOCUMENTATION

10.1 Company API

Method	Endpoint	Purpose
POST	<code>/api/companies</code>	Create company
GET	<code>/api/companies</code>	Get all companies
GET	<code>/api/companies/{id}</code>	Get company by ID
PUT	<code>/api/companies/{id}</code>	Update company
DELETE	<code>/api/companies/{id}</code>	Delete company

10.2 Application API

Method	Endpoint	Purpose
POST	<code>/api/applications</code>	Create application
GET	<code>/api/applications</code>	Get all applications
GET	<code>/api/applications/{id}</code>	Get application by ID
PUT	<code>/api/applications/{id}</code>	Update application
DELETE	<code>/api/applications/{id}</code>	Delete application

10.3 Preparation API

Method	Endpoint	Purpose
POST	<code>/api/preparations</code>	Create preparation
GET	<code>/api/preparations</code>	Get all preparations
GET	<code>/api/preparations/{id}</code>	Get preparation by ID
PUT	<code>/api/preparations/{id}</code>	Update preparation
DELETE	<code>/api/preparations/{id}</code>	Delete preparation

10.4 Interview API

Method	Endpoint	Purpose
POST	<code>/api/interviews</code>	Create interview
GET	<code>/api/interviews</code>	Get all interviews
GET	<code>/api/interviews/{id}</code>	Get interview by ID
PUT	<code>/api/interviews/{id}</code>	Update interview
DELETE	<code>/api/interviews/{id}</code>	Delete interview

11. FRONTEND IMPLEMENTATION

The frontend is developed using React.

The main React components are:

App.jsx
App.css
Companies.jsx
Applications.jsx
Preparation.jsx
Interviews.jsx
TrackerContext.jsx

App.jsx

App.jsx contains the main application layout and dashboard.

Companies.jsx

Handles company management and company search.

Applications.jsx

Handles job applications, searching, and status filtering.

Preparation.jsx

Handles preparation topics, searching, filtering, and progress.

Interviews.jsx

Handles interview records, results, and feedback.

TrackerContext.jsx

Provides shared state between different components and loads data from the backend API.

12. BACKEND IMPLEMENTATION

The backend is developed using Java Spring Boot.

The backend contains:

controller
entity
repository
exception

Entity Layer

Entity classes represent database tables.

The project contains:

- Company
- Application
- Preparation
- Interview

Repository Layer

Spring Data JPA repositories are used for database operations.

Repositories include:

- CompanyRepository
- ApplicationRepository
- PreparationRepository
- InterviewRepository

Controller Layer

REST controllers handle HTTP requests and provide API endpoints.

Controllers include:

- CompanyController
- ApplicationController
- PreparationController
- InterviewController

Exception Layer

The project includes:

- `ResourceNotFoundException`
- `GlobalExceptionHandler`

13. VALIDATION AND EXCEPTION HANDLING

Input validation is implemented in the backend using Jakarta Validation.

Required fields are validated before data is stored.

For example, empty required fields are rejected.

The application also handles missing resources using `ResourceNotFoundException`.

Global exception handling is implemented through `GlobalExceptionHandler`.

The SOP requires validation and exception handling as part of the CRUD application and also recommends server-side validation even when client-side validation exists.

14. TESTING AND RESULTS

Testing was performed at three levels:

1. Frontend Testing
2. API Testing
3. Database Testing

14.1 API Testing

Postman was used to test the REST APIs.

The following operations were tested:

- GET
- POST
- PUT
- DELETE
- Validation
- Resource-not-found handling

The SOP recommends testing each endpoint independently, testing valid and invalid data, checking update/delete IDs, and verifying database values after operations.

Placement-Preparation-Tracker

> .git

> .postman

> backend

> frontend

> postman

README.md

3.1 KB

http://localhost:8080/api/companies

GET

Send

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 6 Test Results

200 OK · 964 ms · 324 B

JSON Preview Visualize

```
1 [
2   {
3     "companyName": "TCS",
4     "id": 2,
5     "industry": "IT",
6     "location": "Chennai",
7     "website": "www.tcs.com"
8   }
9 ]
```

Placement-Preparation-Tracker

> .git

> .postman

> backend

> frontend

> postman

README.md

3.1 KB

http://localhost:8080/api/applications

POST

Send

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

Body Cookies Headers 6 Test Results

200 OK · 1.97 s · 406 B

JSON Preview Visualize

```
1 {
2   "company": "Infosys",
3   "jobRole": "Software Engineer",
4   "applicationDate": "2026-09-17",
5   "status": "Applied",
6   "location": "Chennai",
7   "notes": "Applied through placement portal"
8 }
```

Body Cookies Headers 6 Test Results

200 OK · 1.97 s · 406 B

JSON Preview Visualize

```
1 {
2   "applicationDate": "2026-09-17",
3   "company": "Infosys",
4   "id": 4,
5   "jobRole": "Software Engineer",
6   "location": "Chennai",
7   "notes": "Applied through placement portal",
8   "status": "Applied"
9 }
```

Placement-Preparation-Tracker

> .git

> .postman

> backend

> frontend

> postman

README.md

3.1 KB

http://localhost:8080/api/applications/1

PUT

Send

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

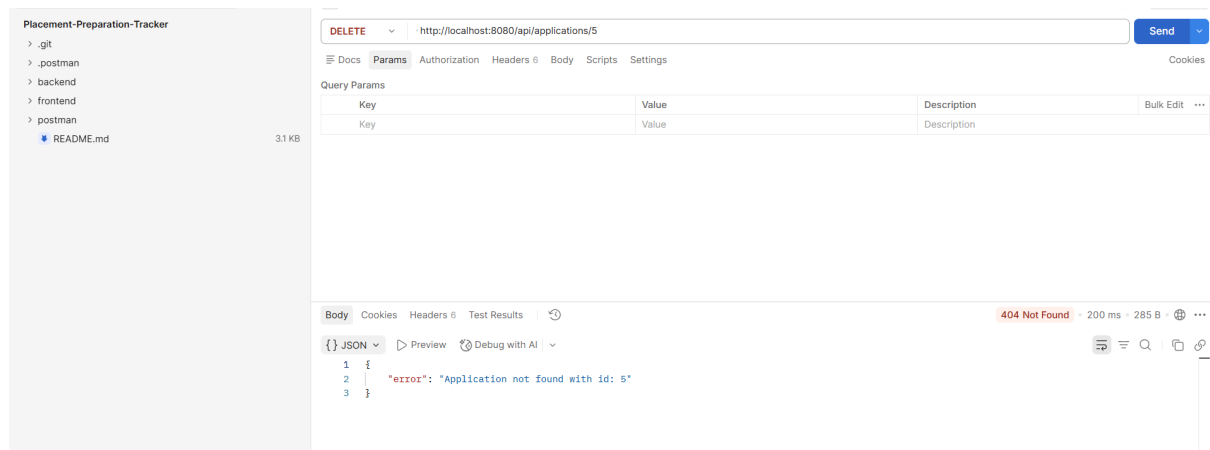
Schema Beautify

Body Cookies Headers 6 Test Results

200 OK · 328 ms · 411 B

JSON Preview Visualize

```
1 {
2   "applicationDate": "2026-09-17",
3   "company": "Infosys",
4   "id": 1,
5   "jobRole": "Senior Software Engineer",
6   "location": "Chennai",
7   "notes": "Updated after technical round",
8   "status": "Selected"
9 }
```



14.2 Frontend Testing

The following frontend functions were tested:

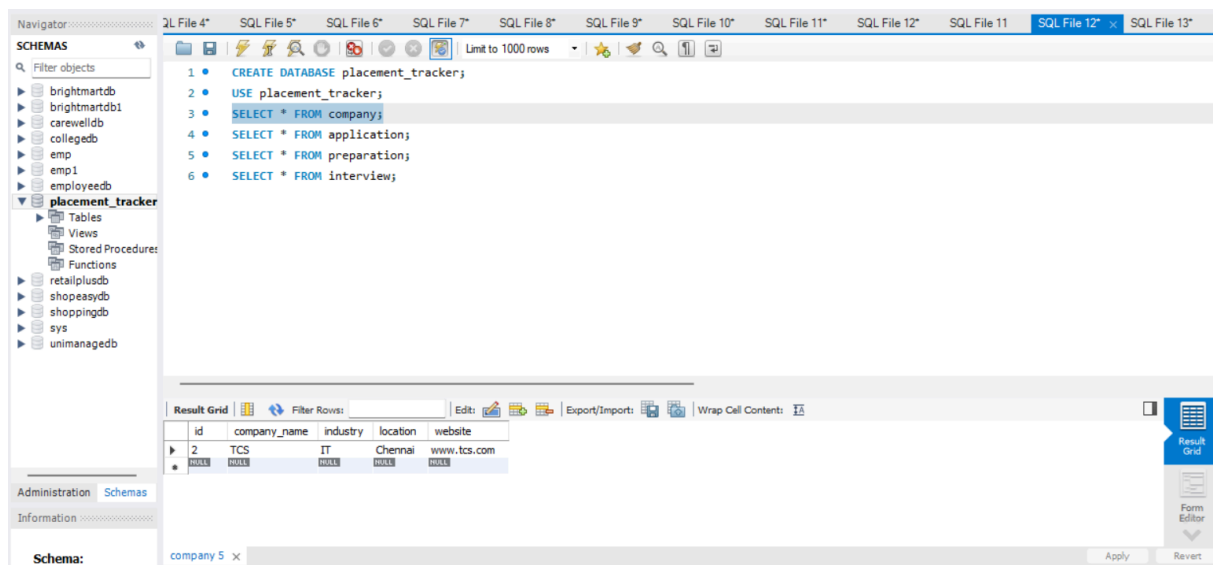
Test	Result
Add Company	Passed
Edit Company	Passed
Delete Company	Passed
Search Company	Passed
Add Application	Passed
Edit Application	Passed
Delete Application	Passed
Search Application	Passed
Filter Application	Passed
Add Preparation	Passed
Edit Preparation	Passed
Delete Preparation	Passed
Search Preparation	Passed
Filter Preparation	Passed
Add Interview	Passed
Edit Interview	Passed
Delete Interview	Passed
Page Refresh / Persistence	Passed

14.3 Database Testing

MySQL Workbench was used to verify the stored records.

The following tables were checked:

- company
- application
- preparation
- interview



15. INSTALLATION AND EXECUTION

Step 1: Create the Database

Open MySQL Workbench and create:

```
CREATE DATABASE placement_tracker;
```

Step 2: Configure Backend

Open:

backend/src/main/resources/application.properties

Configure the database connection.

Step 3: Start Spring Boot

Open a terminal inside the backend folder:

```
.\mvnw.cmd spring-boot:run
```

Backend:

<http://localhost:8080>

Step 4: Start React

Open another terminal inside the frontend folder:

```
npm install  
npm run dev
```

Frontend:

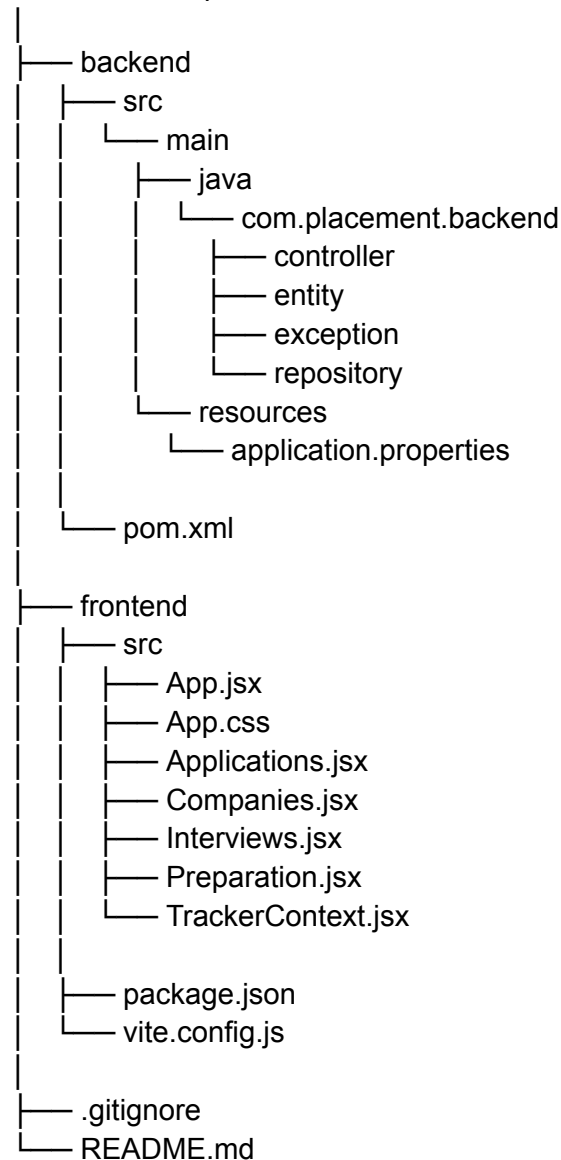
<http://localhost:5173>

Step 5: Open the Application

Open the frontend URL in a browser and use the dashboard and modules.

16. PROJECT STRUCTURE

Placement-Preparation-Tracker



17. CHALLENGES AND SOLUTIONS

Challenge 1: Frontend and Backend Integration

Initially, the frontend and backend were developed separately.

Solution

REST APIs were created in Spring Boot and React was connected to those APIs using HTTP requests.

Challenge 2: Data Persistence

Frontend-only data would disappear after refreshing the browser.

Solution

The application was connected to MySQL through Spring Data JPA and Hibernate so that records are stored in the database.

Challenge 3: Validation

Users may submit incomplete forms.

Solution

Required fields were validated in both the application workflow and the backend using validation annotations.

Challenge 4: Exception Handling

Requests may be made for records that do not exist.

Solution

`ResourceNotFoundException` and `GlobalExceptionHandler` were implemented to return appropriate error responses.

Challenge 5: Shared Data Between Modules

Dashboard values need to reflect changes made in other modules.

Solution

React Context was implemented through `TrackerContext` to share application data between components.

18. GIT AND GITHUB

Git was used as the version-control system for the project.

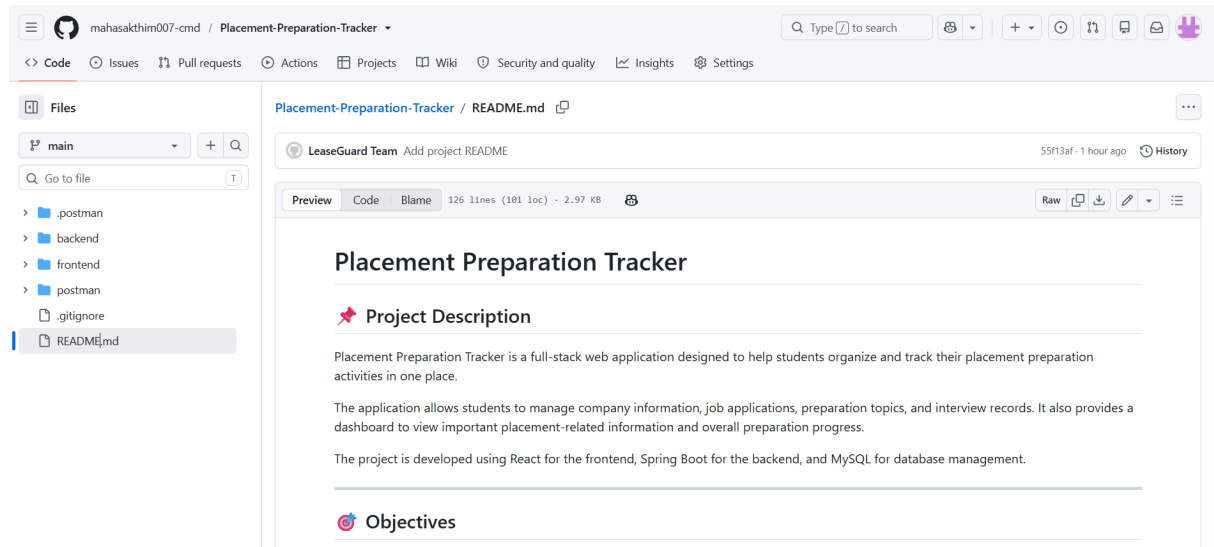
The project was initialized as a Git repository and meaningful commits were created during development.

The completed project was pushed to GitHub.

GitHub Repository

Placement Preparation Tracker

<https://github.com/mahasakthim007-cmd/Placement-Preparation-Tracker>



19. FUTURE ENHANCEMENTS

The following features can be added in future versions:

- Student registration and login
- Authentication and authorization
- Placement preparation reminders
- Email notifications
- Resume management
- Aptitude practice tracking
- Coding practice tracking
- Interview scheduling
- Advanced analytics
- Report generation
- Cloud deployment

20. RESULTS AND BENEFITS

The completed Placement Preparation Tracker provides a centralized system for placement preparation.

The application successfully provides:

- Company management

- Application management
- Preparation tracking
- Interview management
- CRUD operations
- Search functionality
- Filtering functionality
- Dashboard statistics
- Progress tracking
- Input validation
- Exception handling
- REST API integration
- MySQL database persistence
- Postman API testing
- GitHub source-code management

The project demonstrates integration between frontend, backend, REST APIs, and a relational database, which aligns with the SOP's expected full-stack learning outcomes.

21. CONCLUSION

The Placement Preparation Tracker is a complete full-stack CRUD-based web application developed to help students manage their placement preparation activities efficiently.

The project integrates React for the frontend, Spring Boot for the backend, Spring Data JPA and Hibernate for database interaction, and MySQL for persistent storage.

The application provides four major modules: Companies, Applications, Preparation, and Interviews. Each module supports CRUD operations, while additional features such as searching, filtering, validation, exception handling, dashboard statistics, and progress tracking improve the usability of the application.

The project was tested using Postman and MySQL Workbench, and the source code was maintained using Git and GitHub.

The completed application demonstrates the practical implementation of a full-stack web application from frontend development through backend API development and database integration.